Parallel Monte Carlo Option Pricing with PETSc:
HPC reductions, calibrated volatility, and PETSc PDE / Krylov

APMA 4302 — Columbia University
(Daniel David / UNI: dad2232)

May 15, 2026

Abstract

This report documents a deliberately broad scientific computing project that ties together three PETSc-aware threads: **(i)** scalable Monte Carlo estimation with rigorous uncertainty quantification, **(ii)** sparse linear algebra and preconditioned Krylov solvers on representative PDE discretizations, and **(iii)** reproducible visualization of fields and trajectories through ParaView. The financial thread prices European, Asian, and barrier-style options under geometric Brownian motion (GBM) using MPI-parallel path batches and global reductions implemented with PETSc's programming model [2, 3]. The PDE thread follows the *matrix–vector–solver* pedagogy of Bueler's SIAM text [1]: Poisson on structured grids, implicit heat and advection–diffusion time steps, and small diagnostic sweeps over KSP and PC choices. A scalar **SNES** solve recovers implied volatility from Black–Scholes as a clean nonlinear equation example.

The second phase connects the same Monte Carlo engine to **historical SPY data**: rolling realized volatility replaces a toy constant σ, and a roll-forward experiment reprices an ATM-style call along a sparse calendar of market snapshots.

The goal of this project was neither finance nor PDE theory for their own sake: using both as for practicing a single style of work at scale—**PETSc + MPI** programming with partitioned data, global reductions, solver objects, and honestly reported timings. Monte Carlo paths and grid-based PDE steps simply expose *different* dominant costs (sampling vs. sparse linear algebra) while demanding the *same* engineering habits: isolate the numerical kernel, parallelize cleanly, and validate with layered evidence (closed forms, mesh refinement, statistical error decay, and performance curves) [1, 2]. That unification is what large-scale scientific computing increasingly expects from practitioners who must move between kernels without reinventing their software stack each time [3].

Citations overview. Options theory follows Black–Scholes and Merton [4, 5]; PETSc software practice follows Balay et al. [2, 3]; PDE discretization + Krylov/Newton intuition follows Bueler [1]; visualization practice follows Ahrens et al. [6].

Contents

1 Introduction: problem and claims

1.1 The motivating tension

Monte Carlo (MC) option pricing is attractive because it scales well in the number of paths and because it extends to payoffs where closed forms are awkward (e.g. path-dependent Asian averages and barrier knock-out structures). Yet MC is not "free parallelism": one still needs deterministic reproducibility policies, stable variance estimators, and careful interpretation of parallel timings when collectives and load imbalance enter the story.

At the same time, a PETSc-centered numerical methods course emphasizes different bottlenecks: sparse matrix assembly, preconditioned Krylov iterations, and nonlinear solvers [1]. Those kernels look different from MC on the surface, but they share a common engineering spine: partition data, compute locally, reduce globally, measure time and iteration counts honestly [3].

1.2 What we claim this repository demonstrates

We claim three research outcomes supported by artifacts in the report figures and CSV logs:

1. **Statistical correctness and scaling of the MC pricer:** European prices match Black–Scholes at tolerances consistent with standard error budgets; error vs. N follows the expected square-root law on log–log plots; barrier sweeps show how payoff geometry modulates variance; strong/weak scaling curves document parallel behavior for a fixed implementation.
2. **Deterministic PDE pipeline correctness and solver literacy:** Poisson spatial convergence under mesh refinement is consistent with finite-difference expectations; PC and KSP sweeps make preconditioner and outer-solver tradeoffs visible on the same discrete operator; heat and advection–diffusion ParaView exports document time-evolving fields consistent with implicit stepping, and trajectory `.vtp` bundles connect the Monte Carlo thread to the same visualization discipline [6].
3. **Historical realism:** realized volatility from historical SPY data is materially time-varying; roll-forward repricing shows how the *same* MC engine responds when inputs drift with the market rather than remaining stylized constants.

1.3 Where the ideas came from

The financial modeling layer is textbook GBM risk-neutral pricing as presented in foundational derivatives references [4, 5]. The PETSc object model (Vec, Mat, KSP, PC, SNES) and the habit of treating performance as a first-class output follow the PETSc team's user manual and the original PETSc software paper, and class (of course) [2, 3]. The PDE discretization + Krylov + Newton narrative is intentionally aligned with Bueler's *PETSc for Partial Differential Equations* [1], because that book is written precisely for readers who need to connect mathematical definitions to working code: structured grids, residual-based stopping, and preconditioner choices that are not black boxes.

The visualization layer follows standard ParaView practice for scientific vector/tensor data [6].

2 Mathematical models

2.1 Option pricing under GBM

Under risk-neutral GBM,

$$dS_t = rS_t\,dt + \sigma S_t\,dW_t,$$

with an exact exponential integrator over Δt for path simulation, as is standard in Monte Carlo implementations [4, 5]. Payoffs implemented in the codebase include European calls, Asian averages, and barrier knock-out structures; these choices stress different variance profiles and different parallel workload shapes (e.g. barrier paths may terminate early in alternative implementations; here the emphasis is on statistical behavior vs. barrier level sweeps).

2.2 Grid PDEs as the solver-dominated workload

The PDE thread supplies the **sparse linear algebra** side of the project: each model is a standard classroom example from Bueler [1], chosen so the same PETSc objects (Mat, Vec, KSP, PC) appear in settings that differ in symmetry and time structure:

$$\begin{aligned} -\Delta u &= f \quad \text{(Poisson: one elliptic solve, spatial convergence)},\\ u_t - \kappa\Delta u &= g \quad \text{(heat: repeated SPD steps, ParaView movies)},\\ u_t + \mathbf{v}\cdot\nabla u - \nu\Delta u &= s \quad \text{(advection–diffusion: nonsymmetric shifts, parallel-safe PC choices)}. \end{aligned}$$

Structured five-point stencils on the unit square discretize all three; implicit Euler in time reduces the parabolic problems to a linear system per step. The point for this report is not PDE theory but repeatable solver practice: assemble, solve with instrumented KSP, export `.vtp`/`.pvd` for inspection, and record iteration counts alongside wall time [1, 2].

3 Numerical methods and PETSc objects

3.1 Finite differences and Krylov solvers

We use structured five-point Laplacians on the unit square for Poisson, consistent with a first-course finite-difference treatment [1]. Linear systems are solved with PETSc KSP + PC. For SPD Poisson-like operators, CG with Jacobi is a transparent baseline that exposes iteration counts and residual norms without hiding complexity inside a factorization black box [1, 2].

Advection–diffusion in parallel. For parallel runs, incomplete LU as a preconditioner is often unavailable or fragile for distributed sparse matrices; our advection–diffusion driver therefore defaults to **CG + Jacobi** for the implicit step (GMRES + Jacobi is a common alternative when nonsymmetry is more pronounced). This is not merely an implementation detail: it is the same "preconditioners are not magic" lesson emphasized in Krylov coursework and in Bueler's treatment of scalable preconditioning [1, 2].

3.2 Nonlinear solve (SNES) for implied volatility

We recover implied σ from a synthetic market price by solving the scalar equation $\mathrm{BS}(\sigma) = V_{\mathrm{mkt}}$ with PETSc SNES, optionally supplying a hand-coded 1×1 Jacobian (vega). This is a deliberate bridge to the Newton / SNES week: the problem is tiny, but the software pattern is the same as large residual equations [1, 2].

4 Parallel Monte Carlo engine: what was built and how we read the evidence

4.1 Implementation narrative

The Monte Carlo engine partitions path indices across MPI ranks, simulates GBM paths (including optional antithetic pairing for variance reduction), evaluates payoffs, and performs global reductions for the mean, mean-square contributions needed for variance, and valid path counts. This is the same SPMD structure PETSc users apply to partitioned meshes, except the "mesh cells" are replaced by path batches [3].

4.2 Interpreted results (Monte Carlo)

Verification. European MC prices align with Black–Scholes within uncertainty budgets implied by the standard error; this is the first sanity check that parallel RNG partitioning and reductions are consistent [4].

Convergence in N. Log–log plots of error vs. N are the correct statistical diagnostic: we do not expect machine-precision convergence, but we do expect the error to shrink with the inverse square root of independent samples; figures in Section 8 document that behavior.

Barrier geometry. Barrier sweeps illustrate a different "research conclusion": MC difficulty is payoff-dependent. As the barrier moves closer to the spot distribution, estimates become noisier or more sensitive, which is visible in stderr and confidence-interval geometry (Figure 8).

Scaling. Strong and weak scaling curves are not universal truths; they are *measurements* of a particular implementation on a particular machine. The dashboard (Figure 5) is therefore best read as an empirical case study: where speedup bends, whether weak scaling keeps wall time flat, and how sensitive timings are to launcher overhead and work per rank.

5 Ground Truth: why historical volatility is scientifically meaningful

A constant σ is a modeling convenience, not a claim about markets. Phase 2 replaces that convenience with rolling realized volatility estimated from log returns over a trailing window of trading days, then feeds the resulting $\sigma_{\text{roll}}(t)$ into the same MC pricer used in Phase 1.

This connects the project to data conditioning and nonstationarity: the volatility input is now path-dependent on the market history, which is closer to how practitioners think about risk metrics than a single hard-coded constant.

Figure 9a shows the input regime: SPY's level and the rolling annualized σ spike around stress periods (e.g. 2020), which is exactly the kind of "volatility as a signal" narrative implied by realized-vol estimators. Figure 9b shows roll-forward snapshots: σ and repriced option values move with the calibration date. The twin-axis design is deliberate: spot and option prices live on different magnitudes, and forcing them onto one axis would visually erase the option story.

6 PETSc PDE results: verification and solver sweeps as small experiments

6.1 Poisson as a correctness anchor

Poisson spatial convergence (Figure 10) is the deterministic anchor that says: before interpreting fancy plots, the FD + PETSc stack reproduces the expected refinement behavior on a classical elliptic model [1].

6.2 PC and KSP sweeps as comparative studies

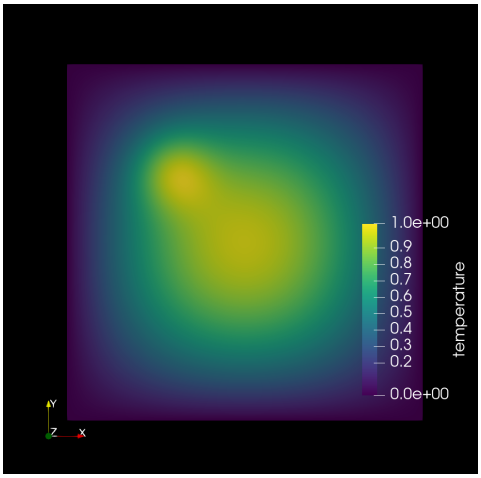
The PC sweep (Figure 13) and KSP sweep (Figure 14) are intentionally modest comparative experiments: they record iteration counts and wall times under controlled changes to solver configuration. The research conclusion is not "Jacobi always wins"—it is that solver performance is empirical, and PETSc makes such empirical questions cheap to ask once the discretization is in place [1, 2]. On coarse grids, iteration counts can collapse to a single iteration; finer meshes separate methods more clearly, which is itself a useful student observation.

6.3 Strong scaling as a performance case study

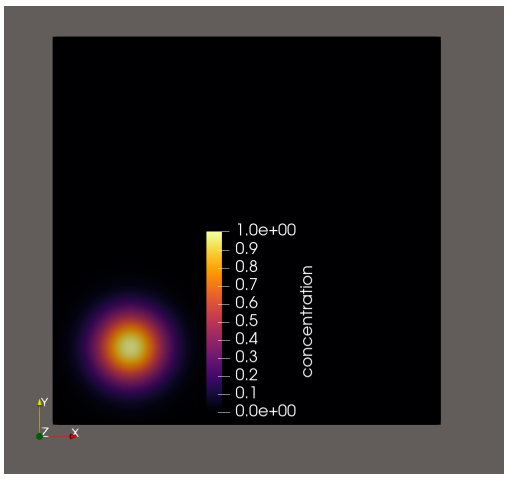
The Poisson strong-scaling figures and summary table document runtime variability and diminishing returns at higher rank counts for a fixed discrete problem. That pattern is a legitimate HPC conclusion: parallel efficiency is a function of problem size, communication cost, and the quality of load balance [3].

7 Time-dependent fields and trajectory visualization (ParaView)

This section embeds representative ParaView (or ParaView-equivalent) stills directly in the PDF: PDE snapshots (Figures 1a–1b), Monte Carlo and enriched trajectory bundles (Figures 2–4), and the synthetic MPI batch metaphor (Figure **??**). The broader point—consistent with Bueler's emphasis on interpretable outputs and with ParaView's role in large-scale visualization—is that **solvers and samplers should produce artifacts that humans can interrogate**, not only scalar timings [1, 6]. Regenerate the underlying `.vtp`/`.pvd` files with the commands in `PROJECT_OUTLINE.md` Appendix A whenever parameters change; keep `overleaf/figures/` in sync before submission (Section 11.4). The trajectory stills connect the Monte Carlo story to geometry: how batching, barriers, and pathwise risk summaries appear once paths are embedded in a viewable (t, S, z) space rather than remaining only as CSV columns.



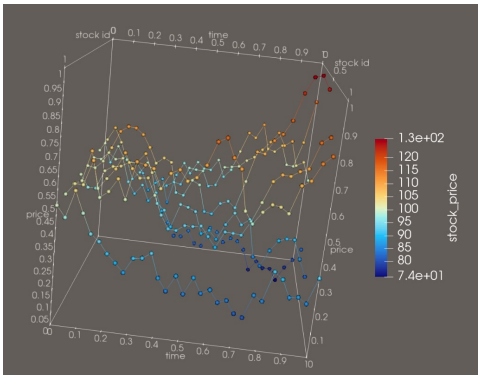
(a) Heat equation at an early time: this frame exists to show that the implicit Euler + five-point Laplacian discretization produces a smooth, physically plausible temperature field on the unit square. It matters because it is the *visual* counterpart to the iteration/residual logs: if the field were jagged or violated boundary conditions, the solver diagnostics would be harder to trust. The bright interior and darker boundary banding reflect the chosen initial bump and homogeneous Dirichlet boundaries—exactly the qualitative behavior expected before studying later-time diffusion.



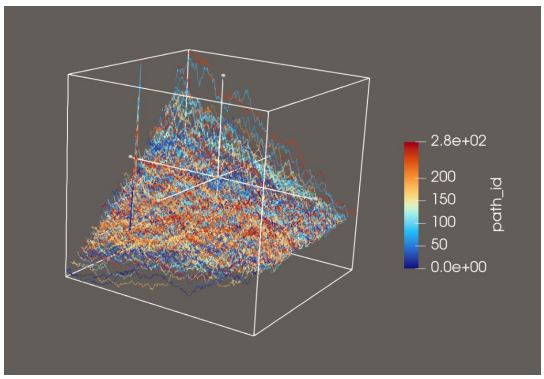
(b) Advection–diffusion concentration: this snapshot ties the nonsymmetric implicit step (upwind-like advection + diffusion) to a spatial pattern you can *see*. It is important because it makes the course's warning about parallel ILU vs. Jacobi tangible: the field is the outcome of the same Krylov solve whose iteration counts we later sweep. The "hills" and drifted structure read as combined transport and spreading—if the velocity field were turned off, the plume would relax toward a simpler diffusion patch.

Figure 1: Representative PETSc-driven PDE snapshots exported to VTK surface polylines (`.vtp`) and opened in ParaView. Together they support the narrative that PETSc workloads are not only `KSP` counters: they are fields whose shape can be checked by eye against boundary conditions and qualitative stability.

8 Figures: Monte Carlo and HPC dashboards

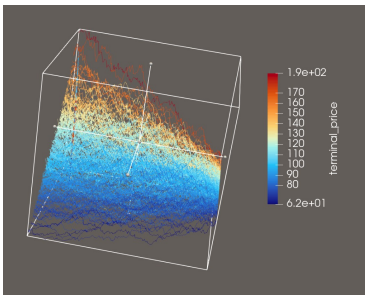


(a) GBM `path_bundle.vtp` (Monte Carlo exporter): lines are individual simulated paths embedded in a normalized (t, S, z) frame so the camera is not dominated by a single long axis. Coloring by `stock_price` highlights where paths explore in price space over time. This matters because it is the bridge claim of the report in miniature—**many independent samples** of the same Markovian dynamics—and the rainbow "feathers" read as ensemble dispersion: wider color spread at later times corresponds to log-normal variance growth.

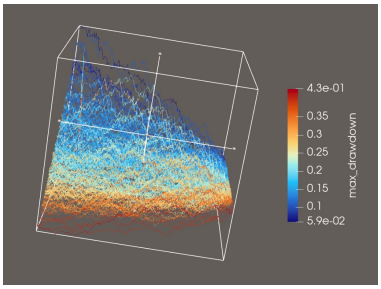


(b) Same geometry with `path_id` as the scalar: each path is a constant hue along its polyline, which makes the **parallel batch structure** obvious (paths are indexed lanes). It supports the MPI narrative later—before ranks exist in PETSc, we already partition work by path index. The dense bundle shape should look like a correlated forest, not a surface: if it looked like a triangulated sheet, that would indicate a wrong ParaView representation (e.g. triangulating points as a surface).

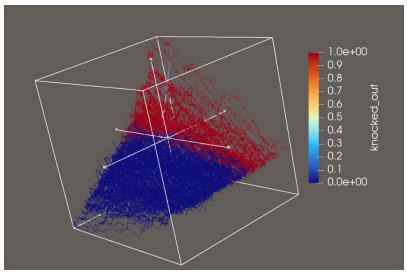
Figure 2: Monte Carlo path bundle (`path_bundle.vtp`): two equivalent views of the same polylines, emphasizing either physical economics (`stock_price`) or implementation partitioning (`path_id`).



(a) `terminal_price` (per-path terminal level, replicated along the polyline) encodes where each path ends in spot space; warm colors are higher terminal draws. It links the time-sliced view to the **payoff input** that actually drives European-style summaries.



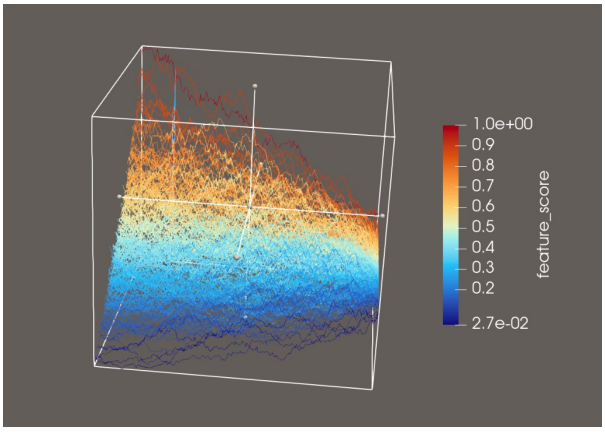
(b) `max_drawdown` summarizes pathwise depth of drawdowns; larger values (warmer) are paths that dipped far below their running maximum before recovering. It matters for risk narratives beyond the terminal value alone.



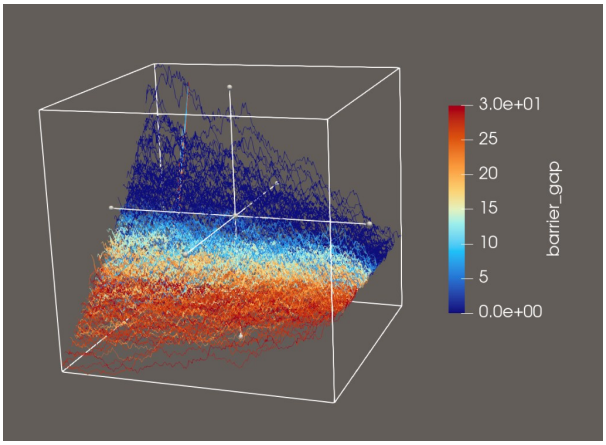
(c) `knocked_out` is essentially binary here: paths that touched the barrier level are one color, survivors another. The split should read as **geometry interacting with a threshold**, analogous to barrier option knock-out events.

Figure 3: Enriched GBM export (`stock_paths.vtp`): pathwise summaries exposed as point fields so ParaView can color tubes/lines without CSV gymnastics. These panels answer "what does the model *do* to paths?" beyond mean/variance reduction alone.

9 Figures: Historical inputs and roll-forward



(a) `feature_score` is a deliberately engineered scalar combining normalized terminal level, drawdown, and barrier gap (see repository exporter). Smooth gradients mean the chosen normalization is spreading paths across $[0, 1]$; a degenerate flat image would mean all paths look the same under those features.



(b) `barrier_gap` measures how far a path's running maximum stayed *below* the barrier (in price units, floored at zero). Large gaps (warm) mean "never came close"; small gaps mean near-touching—the visual complement to knock-out indicators.

Figure 4: Composite scalars on `stock_paths.vtp`: these are not standard textbook fields; they are **reporting aids** that compress several path statistics into a single colormap for storytelling.

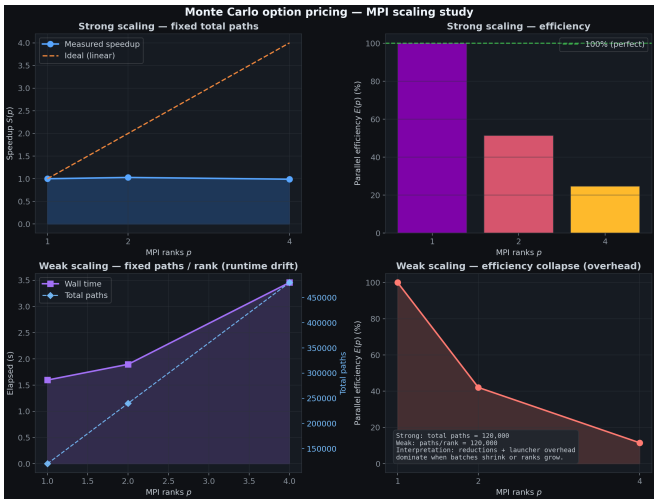


Figure 5: HPC scaling dashboard: this figure compresses strong and weak scaling experiments into one glanceable page. It summarizes timings and speedups from the repository's scaling drivers. Parallel MC is not automatically linear speedup—collectives, load balance, and fixed problem size all bend the curves. Near-linear segments mean additional ranks buy wall time; flattening or downturns mean communication or insufficient work per rank dominates.

10 Figures: Poisson verification, sweeps, and scaling

11 Research conclusions

11.1 Monte Carlo conclusions

The Monte Carlo stack is **correct in expectation** (European vs. BS), shows **statistical convergence** consistent with $O(N^{-1/2})$ error decay, and exhibits **payoff-driven variance** in barrier

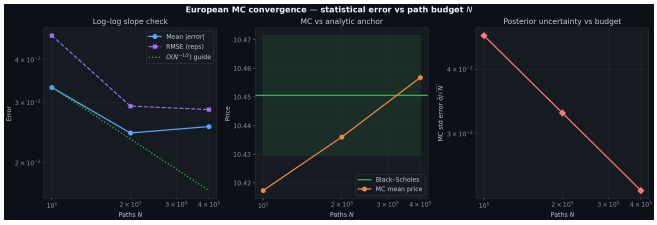
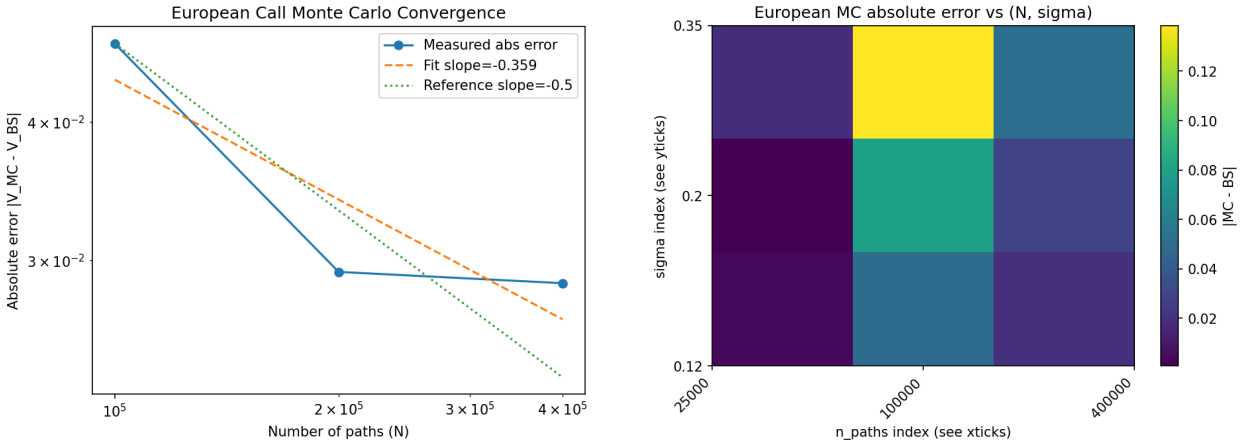


Figure 6: Monte Carlo convergence triptych vs. Black–Scholes: each panel fixes a viewpoint (price error, stderr, relative error) while varying N or method knobs. It is the statistical counterpart to the PDE mesh-refinement story—we expect $O(N^{-1/2})$ decay of Monte Carlo error, not machine precision. Curves that flatten early usually mean the trial stopped before the asymptotic regime or that variance reduction changed the prefactor.



(a) Monte Carlo error vs. path count N on log–log scales. A line near slope $-1/2$ is the signature of square-root Monte Carlo error decay; systematic offsets between markers usually mean different RNG streams, antithetic pairing, or payoff implementations—not "better physics" by themselves.

(b) Absolute error over (N, σ): this surface makes the **coupled** sensitivity to both sample count and volatility visible. Ridges or hot spots flag regimes where either sampling is insufficient or σ moves the payoff into a harder region (e.g. near the money with large diffusion).

Figure 7: European Monte Carlo convergence and absolute error comparison against Black–Scholes: together the panels connect *statistical* error budgets to a two-parameter experimental sweep, mirroring how practitioners explore both discretization and sampling uncertainty.

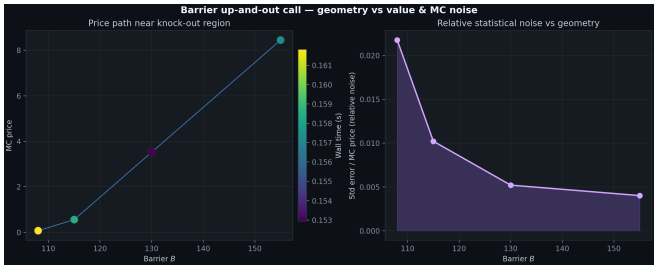
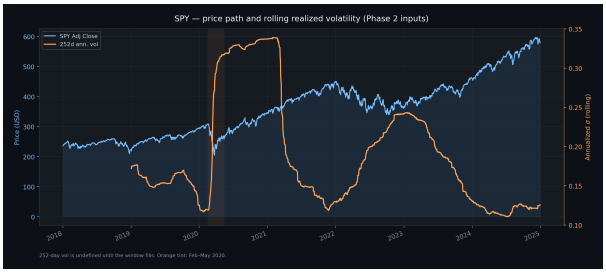
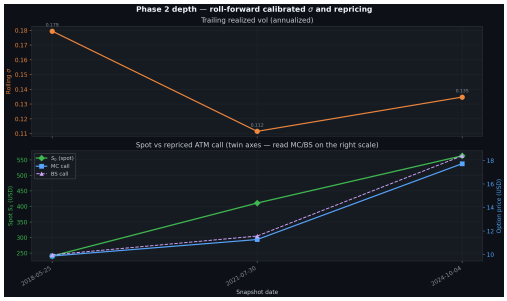


Figure 8: Barrier sweep storyboard: price, stderr, and geometry vs. barrier level. This is the payoff-driven variance story in visual form—as the barrier moves through the bulk of the terminal distribution, estimators often become noisier or more sensitive. A widening stderr band without a corresponding price shift would suggest "statistical stress" rather than a change in the limit value.

regimes. Parallel scaling is **empirical**: it summarizes this implementation on this hardware and



(a) SPY price path with rolling 252-day annualized realized volatility. **Context:** Phase 2 replaces constant σ with a data-driven process. **Why it matters:** it shows **nonstationarity**—the same MC engine will ingest different vol inputs on different roll dates. **How to read it:** volatility spikes (right axis) should align visually with fast market moves (left axis); a flat vol line would mean the rolling window lost responsiveness or the data feed is incomplete.



(b) Roll-forward calibrated σ and repriced call (twin axes). **Context:** each roll date refreshes the volatility input and re-runs pricing. **Why it matters:** it proves the "same code, moving inputs" story for the report's Phase 2 claim. **How to read it:** parallel movements in call value and σ indicate the dominant sensitivity is vega/vol; divergences would point to moneyness or rate effects creeping in through the pricing surface.

Figure 9: Phase 2 market input and roll-forward repricing summary.

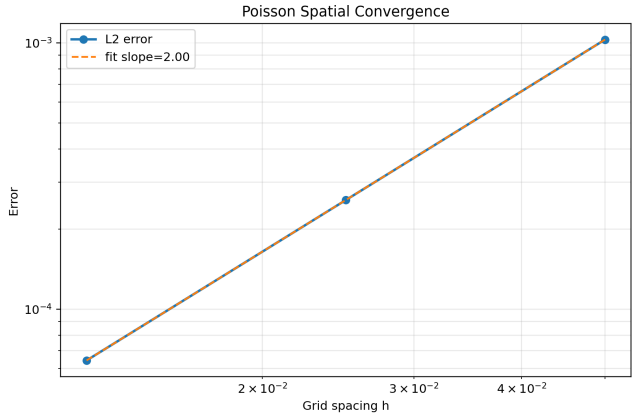


Figure 10: Poisson L^2 error vs. mesh spacing on log–log scales. **Context:** this is the deterministic verification anchor for the PETSc FD stack. **Why it matters:** without this plot, solver sweeps are "moving numbers on a broken model." **How to read it:** a line whose slope matches the discretization order indicates the implementation matches theory; curvature or scatter at the coarsest end often means the asymptotic regime has not been reached.

Ranks	Count	Mean time (s)	Std (s)	Speedup	Efficiency
1	6	0.02453	0.00188	1.00	1.00
2	5	0.01620	0.00237	1.51	0.76
4	5	0.01348	0.00448	1.82	0.45
8	5	0.04346	0.03091	0.56	0.07

Table 1: Example Poisson strong-scaling summary (repository snapshot): counts record independent timing trials; mean and std summarize variability; speedup uses the single-rank mean as baseline; efficiency is speedup divided by ranks. A row where speedup drops below one is a faithful record that **more ranks hurt** for that fixed problem size on that machine—not a plotting bug.

should be quoted with that caveat.

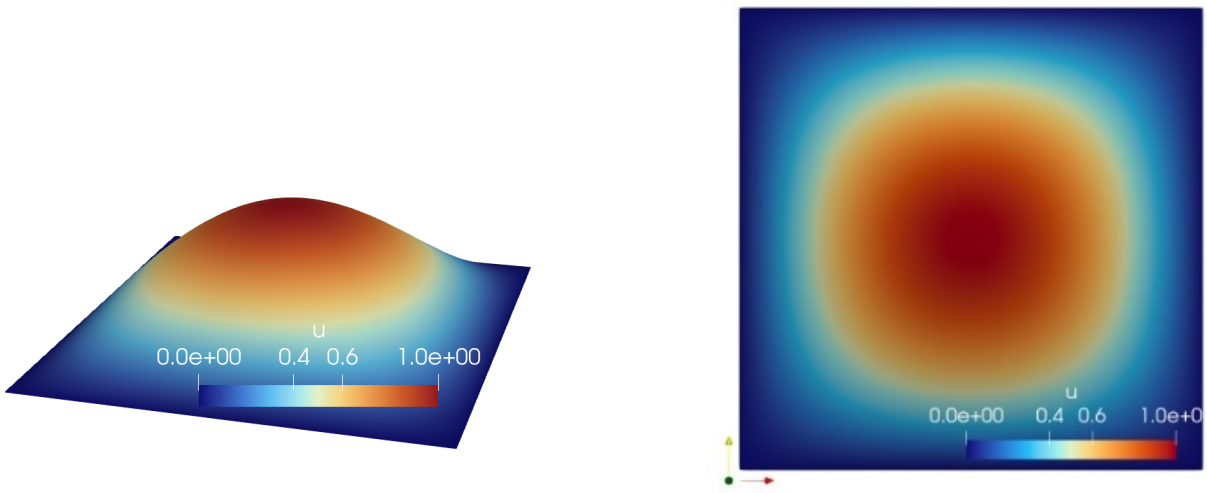


Figure 11: Poisson solution surface and contour/slice visualization. **Context:** elliptic solve on the unit square with manufactured forcing. **Why it matters:** it is the "shape check" complement to the log–log error curve. **How to read it:** smooth extrema and symmetric contour levels indicate a healthy discrete solution; oscillatory or asymmetric patterns would hint at assembly bugs, wrong boundary tags, or inadequate resolution.

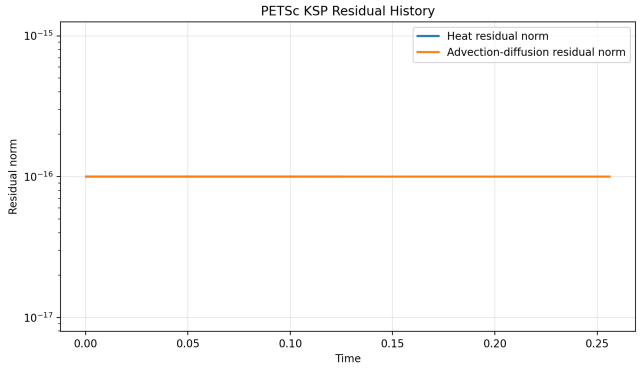


Figure 12: Representative Poisson residual / solver monitoring figure. **Context:** KSP residual norm vs. iteration (or a related diagnostic). **Why it matters:** it documents that the linear solve actually converged and how aggressively the preconditioner reduces modes. **How to read it:** steady geometric decay is typical for CG on SPD problems; stagnation or spikes would prompt a PC change—matching the sweeps in Figures 13–14.

11.2 PDE / Krylov conclusions

Poisson mesh refinement provides the deterministic credibility layer. Solver sweeps support the broader lesson from Bueler [1] and PETSc documentation [2]: **algorithmic choices matter as much as "big-N parallelism"** for wall time and iteration counts.

11.3 Hisstorical Data Conclusions

Realized volatility is time-varying and event-sensitive; roll-forward repricing shows that **identical MC machinery** yields different economic outputs when inputs are disciplined by data rather than

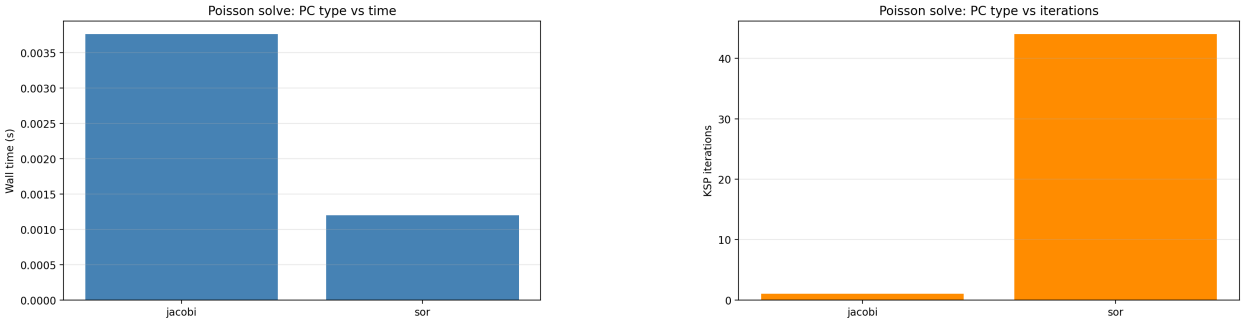


Figure 13: Poisson PC sweep: wall time and iterations vs. preconditioner choice. **Context:** controlled experiment on the *same* discrete operator. **Why it matters:** it reinforces that "parallel PETSc" is not only about ranks—algorithmic preconditioning dominates for ill-conditioned systems. **How to read it:** if time and iterations move together, most cost is Krylov work; if time improves while iterations worsen, the preconditioner may be doing more expensive local work per step.

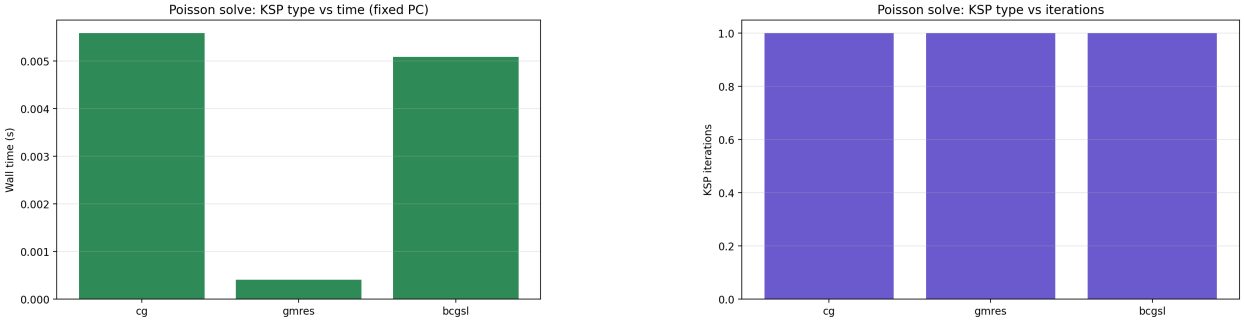


Figure 14: Poisson KSP sweep: wall time and iterations vs. Krylov method. **Context:** outer solver choice for the same SPD system. **Why it matters:** CG vs. GMRES vs. other methods differ in memory and stability on nonsymmetric extensions. **How to read it:** on pure Poisson, CG should be competitive; large GMRES iteration counts on a nearly SPD problem can signal unnecessary nonsymmetric Krylov overhead.

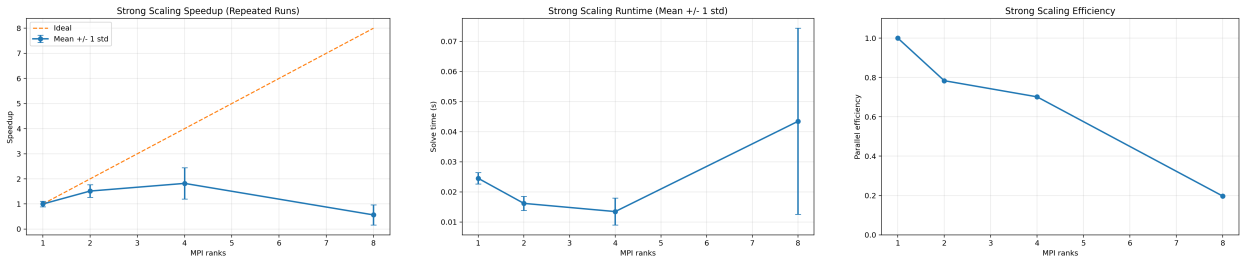


Figure 15: Strong scaling speedup with error bars. Figure 16: Strong scaling runtime variability by rank count. Figure 17: Strong scaling efficiency vs. ranks.

by stylized constants.

11.4 Next steps: low-hanging fruit

The items below are the easiest wins after freezing the current results—mixing submission polish with extensions that reuse existing scripts rather than new theory:

- **Course-bridge scripts:** run or explicitly defer Vec demo, SNES implied vol, and sampling-vs.-solver benchmark; cite outputs or JSON paths if required.
- **Optional quote comparison:** fill `data/raw/quotes_template.csv` and run `compare_mc_to_quotes.py` if market-vs.-model comparison is expected.
- **Larger scaling sweeps:** repeat Poisson/MC timing trials with error bars (weak-scaling PNGs under `visuals/` are available but not yet in this PDF).
- **Solver / time-integration knobs:** try GAMG or other PCs where available; compare TS-based vs. backward-Euler stepping on the parabolic drivers.
- **Final compile:**

11.5 Closing statement

This project is worthwhile because it forces an integrated narrative: the same software ecosystem that supports scalable MC also supports the classical PDE and solver coursework that PETSc was built to serve [1, 2]. The "good research conclusion" is not a single number—it is an evidence chain linking **verification**, **statistical scaling**, **solver empirics**, and **data-conditioned inputs**, each interpreted with the correct reference class and citation lineage.

Software acknowledgment. This work relies on PETSc [2, 3] and `petsc4py`.

References

[1] E. Bueler. *PETSc for Partial Differential Equations: Numerical Solutions in C and Python*. SIAM, Philadelphia, PA, 2020.

[2] S. Balay et al. PETSc/TAO Users Manual, Revision 3.24. Technical Report ANL-21/39, Argonne National Laboratory, 2025.

[3] S. Balay, W. D. Gropp, L. Curfman McInnes, and B. F. Smith. Efficient management of parallelism in object oriented numerical software libraries. In E. Arge, A. M. Bruaset, and H. P. Langtangen, editors, *Modern Software Tools for Scientific Computing*, pages 163–202. Birkhäuser Boston, 1997.

[4] F. Black and M. Scholes. The pricing of options and corporate liabilities. *Journal of Political Economy*, 81(3):637–654, 1973.

[5] R. C. Merton. Theory of rational option pricing. *The Bell Journal of Economics and Management Science*, 4(1):141–183, 1973.

[6] J. Ahrens, B. Geveci, and C. Law. ParaView: An end-user tool for large data visualization. In C. D. Hansen and C. R. Johnson, editors, *The Visualization Handbook*, chapter 36. Elsevier, 2005.